[VERSION: v0.08] [MODULES: 89+] [SYSTEM: ACTIVE]

DEVKiTZ™ Ecosystem & DkZ Dashboard Blueprint

The Architecture of a Vanilla, AI-Governed Development Universe

The Core Manifesto

Absolute Independence

100% Vanilla HTML5, CSS3, and ES6+ JavaScript. Zero external framework dependencies. No React, no Tailwind, no build-step bloat.

`TECH: VANILLA ES6+`

Zero Data Loss

Immutable knowledge architecture. Every task, plan, and walkthrough is rigidly preserved in a dual-database system to ensure context survives across all sessions.

`ARCH: IMMUTABLE DUAL-DB`

AI-Governed Precision

Development driven by the BMAD Methodology (Blueprint, Mapping, Analysis, Design) and executed via a strict 7-agent cybernetic workforce.

`METHOD: BMAD / 7-AGENTS`

Offline-First Agility

Core operations run entirely locally using localStorage and JSON AST as the single source of truth, with seamless Puter Cloud synchronization when online.

`OPS: LOCALSTORAGE / SYNC`

Ecosystem Scale & Topography

Developer Tools [15]
json-formatter, regex-tester, code-differ, api-tester...

Content & Text [8]
markdown-gen, changelog-builder, text-to-speech...

Analysis & Data [7]
ecosystem-analyzer, llm-cost-board, doc-engine...

Backend & System [7]
iceberg, system-check, loop-dashboard...

Design & UI [6]
color-picker, blog-designer, image-forge...

Productivity [6]
timer-tools, tasker, split-browser...

Builder Suites [5]
action-builder, agent-builder, workflow-builder...

AI & Prompts [5]
ai-chat, prompt-generator, prompter...

A unified toolset of 89+ independent modules, governed by a single master blueprint.

The DkZ™ Design System v2

1

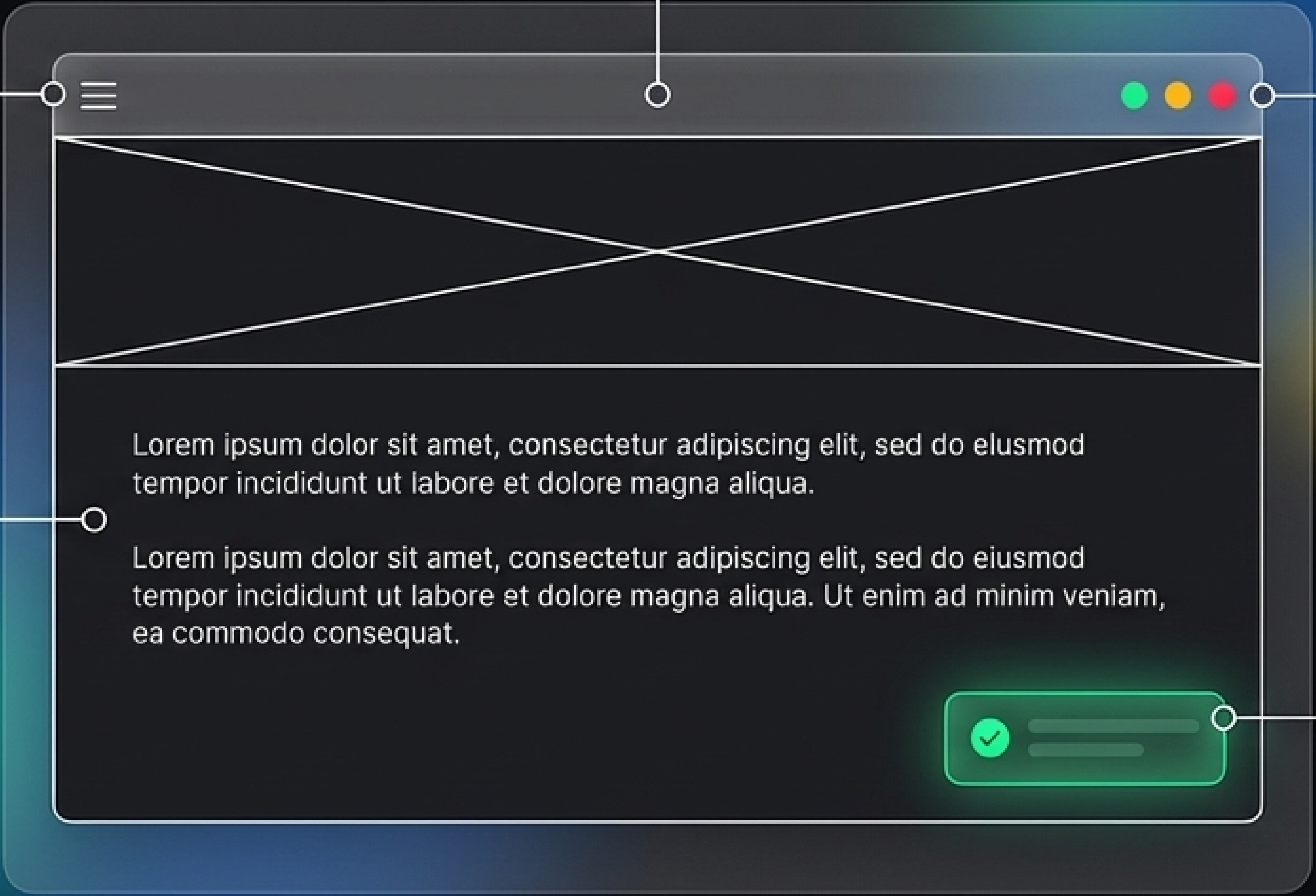
Glassmorphism Header

Mandatory sticky header using `backdrop-filter: blur(24px)`

2

Unified Navigation

Fixed Hamburger Menu triggering a 300px animated slide-in panel connected to 86+ files.



3

Traffic Light Status Palette

`#00ff88` `#FF8800` `#fa1e4e`

Exact hex enforcement.

Green = Perfect.
Yellow = Review.
Red = Offline.

4

Typography Engine

Inter for readable UI text; `JetBrains Mono` for all code, data, and numerical metrics.

5

Toast System

Bottom-right green notifications for all successful offline operations.

The Master Pro Builder Stack

DkZ CurrentVanilla (Frontend)

HTML5
(Semantic Layout / Shadow DOM Isolation)

Vanilla CSS
(Live Theming via CSS Variables, Native Grid)

Vanilla JS
(Native APIs, Proxy State Observers)

Hybrid Backend

Go:
Primary workhorse. High-performance APIs and Apache Iceberg server.
[Status: Green]

Python:
Export compilers and calculations. Acts as the mandatory R23 Fallback for every Go script.
[Status: Yellow]

Node.js:
Real-time WebSockets and JSON I/O.

PHP:
Legacy support, REST CRUD, and SQL sessions.

Database Hierarchy & Priority

1. **JSON** (Priority 1) : Single Source of Truth. Heavily preferred for all core configurations.
2. **Google Sheets** : Used exclusively for cloud collaboration and shared tabular data.
3. **DuckDB** : Initial entry point for analytics and structured queries.
4. **Apache Iceberg** (via Go/Python) : The Knowledge Lake and massive structural catalog.
5. **pgvector** (PostgreSQL) : Storing AI vectors alongside relational data.
6. **MongoDB Atlas** : Deployed only when strictly necessary for remote Vector Search.
7. **ChromaDB** : Reserved for native DB needs, LangChain integrations, and local prototyping.

The DkZ Hub & Puter Cloud Integration

Local DkZ Ecosystem

`● Status: Active (Offline Mode)]`

Scans the modules/ folder natively. Runs entirely offline via localStorage and JSON AST.

Translation Bridge:
Real-time State Mirroring

Puter Open Source Cloud

`[Status: Connected (Synchronized)]`

Translates local states to cloud states without traditional AWS/GCP infrastructure.

API Handshakes

Functional Area	Puter Cloud Interface (via DkzPuiter SDK)
Auth	`DkzPuter.signIn()` (Puter Account mapping)
Key-Value Storage	`DkzPuter.kvSet()` (Mirrors localStorage to Cloud)
File System	`DkzPuter.saveFile()` (Writes directly to /dkz/ cloud directory)
Hosting	`DkzPuter.deploy()` (Pushes static sites instantly to *.puter.site)

Infallible Memory: Triple-Anchoring

Rule §29: The Context Pipeline

JetBrains Nono

Work Artifact

(Task, Walkthrough, or Blueprint)

Anchor 1: Apache Iceberg

(Immutable Data)

The file is logged in catalog.json with metadata sidecars. It is immutable and strictly forbidden from being deleted.

Anchor 2: DkZ Hub

(Searchable Interface)

The artifact becomes a searchable, filterable HTML card within the Hub UI, tagged by module, date, and type.

Anchor 3: AI Copilot

(Active Context)

The artifact is ingested via dkz-copilot.js, granting the AI instant read-access to answer 'How was X built?' during future coding loops.

Nothing is lost. Everything is anchored. Context drift is eliminated.

The Dual-Database Reality

WissenHub™ `JetBrains Mono`

The structured catalog. Contains exact Blueprints, Implementation Plans, and Walkthroughs.

Governed by Apache Iceberg manifests. (Currently 14 massive core entries).

Research Archive (MOD-082) `JetBrains Mono`

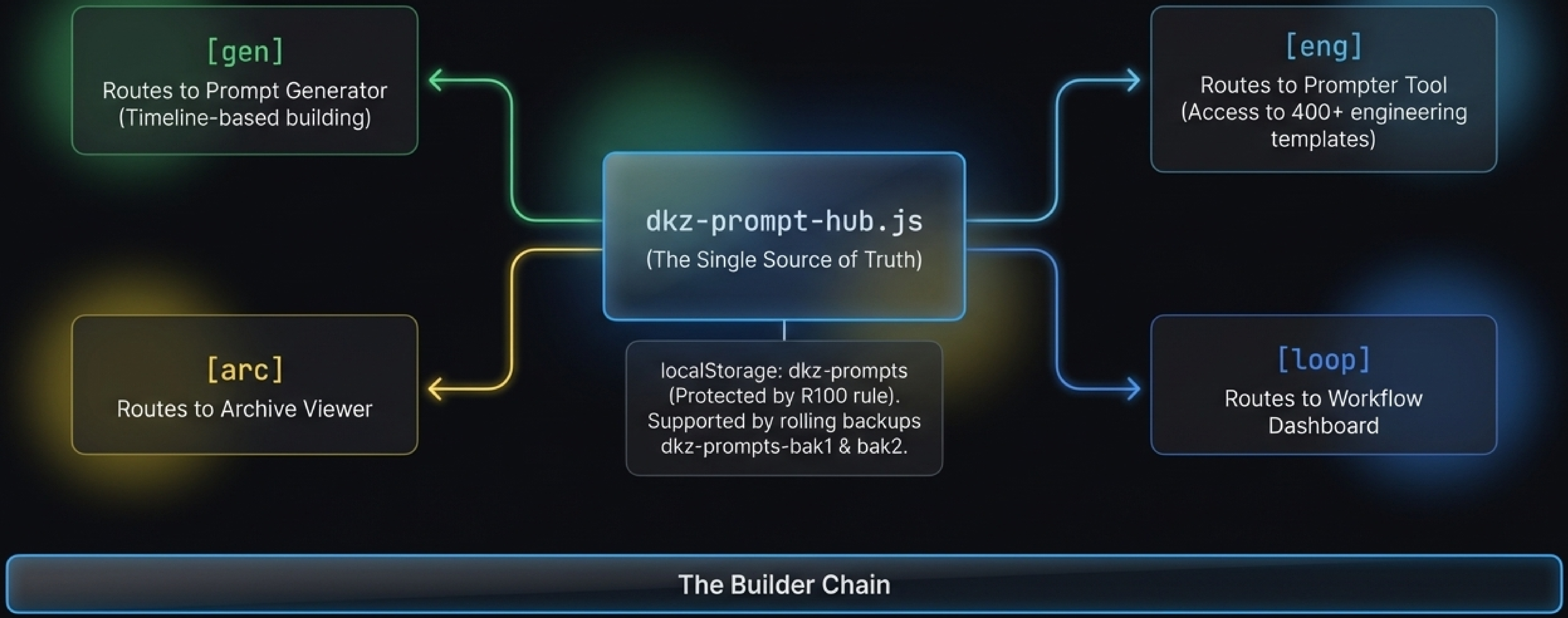
The raw artifact dump. Captures everything from every session (225+ artifacts, 3,600+ screenshots, 91+ conversations).

Powered by DuckDB analytics.

The Actuality Check Rule (Document Decay Lifecycle)

< 7 Days	7 – 30 Days	> 30 Days
Green Status: Current & Valid	Yellow Status: Review Recommended	Red Status: Outdated - Update Mandatory before new code is written

Unified Prompt Architecture



The Prompt Hub powers a localized transfer chain, injecting navigation automatically across 14 builder modules (Action → Skill → Agent → Workflow).

The BMAD Methodology (AI Governance)

"Specialization eliminates hallucinations."

Blueprint

Defining the architectural limits and strict directory structures (`plan.md`).

Mapping

Breaking the blueprint down into atomic, traceable user stories and specs (`spec.md`).

Analysis

Evaluating the existing ecosystem to ensure no redundant modules are built and all vanilla constraints are met.

Design

Executing the exact, scoped task without hallucinating external libraries or unauthorized framework injections.

The 7-Agent Workforce

1. James™ (Guardian) :
Monitors the Constitution. Never writes code. Triggers R24 ALARMS if archiving protocols or core rules are violated.

2. DkZ PM™ :
Writes `spec.md`. Defines user stories and acceptance criteria.

3. DkZ Architect™ :
Writes `plan.md`. Defines the tech stack and atomizes tasks into `tasks.json`.

4. DkZ Developer™ :
The executor. Writes code precisely to the plan.

5. DkZ Reviewer™ :
Validates constraints (Are CSS vars used? Are inputs escaped? Is `features.json` updated?).

6. DkZ Tester™ :
Validates against `tasks.json` and reports bugs.

7. DkZ Documenter™ :
Records learnings to `AGENTS.md` and maintains the Wiki.

Operational Execution: The Ralph-Loop™

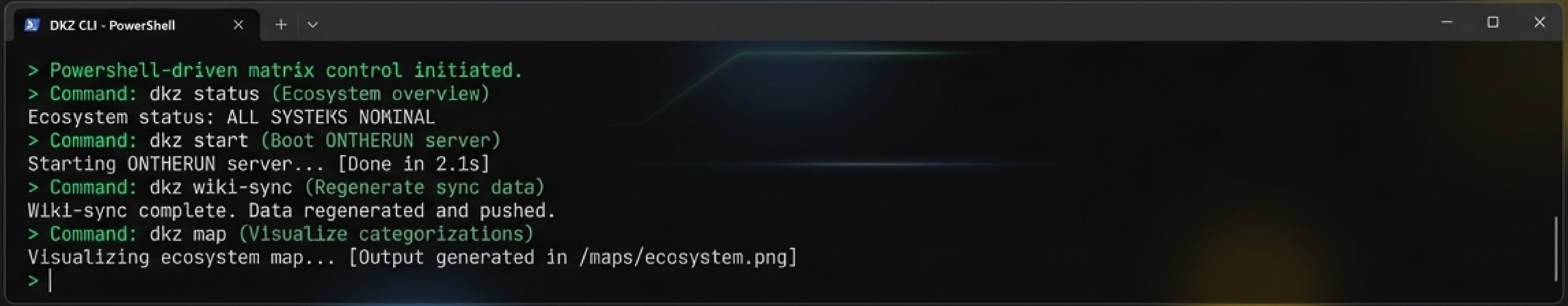
Ubiquitous Access: DkZ Chrome Hub & CLI

DkZ™ Hub Chrome Extension (§30)

The single authorized exception to the Vanilla Only rule.

Built in **React 18** / **Tailwind 3** to meet **Extension Manifest V3** constraints, integrating directly into the browser workflow.

DKZ CLI Terminal (§38)



Synthesis: The Ultimate Developer Engine

BMAD Agentic Loops

Specialized agents reading immutable memory to write vanilla code in perfect, endless cycles.

Vanilla Architecture

Zero framework dependencies ensure code never rots, modules never conflict, and the UI remains instantly responsive.

Triple-Anchored Memory

The Hub, Iceberg, and Copilot ensure the system never forgets a decision, eliminating the classic AI problem of context loss.

Synthesis Takeaway: By enforcing authoritarian technical constraints and immutable data anchoring, DEVkitZ transitions from a simple dashboard into an autonomous, self-sustaining development ecosystem. It doesn't just host tools; it builds itself.